CEFET/RJ

DAL & Comunidade

DATA FRAMES

Eduardo Ogasawara
eogasawara@ieee.org
https://eic.cefet-rj.br/~eogasawara

Exemplo

```r
weight <- c(60, 72, 57, 90, 95, 72)
height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
subject <- c("A", "B", "C", "D", "E", "F")
```

Criando uma tabela básica

```
d <- data.frame(weight=weight, height=height, subject=subject)
head(d)
```

A data.frame: 6 × 3

	weight	height	subject
	<dbl>	<dbl>	<chr>
1	60	1.75	A
2	72	1.80	B
3	57	1.65	C
4	90	1.90	D
5	95	1.74	E
6	72	1.91	F

Adicionando uma coluna

```
d$bmi <- d$weight/d$height^2
head(d)
```

A data.frame: 6 × 4

	weight	height	subject	bmi
	<dbl>	<dbl>	<chr>	<dbl>
1	60	1.75	A	19.59184
2	72	1.80	B	22.22222
3	57	1.65	C	20.93664
4	90	1.90	D	24.93075
5	95	1.74	E	31.37799
6	72	1.91	F	19.73630

Removendo uma coluna

```
d$subject <- NULL
head(d)
```

A data.frame: 6 × 3

	weight	height	bmi
	<dbl>	<dbl>	<dbl>
1	60	1.75	19.59184
2	72	1.80	22.22222
3	57	1.65	20.93664
4	90	1.90	24.93075
5	95	1.74	31.37799
6	72	1.91	19.73630

Carregando uma tabela a partir de um CSV

```r
wine = read.table(
  "http://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data",
  header = TRUE, sep = ",")
head(wine)
```

A data.frame: 6 × 14

	X1	X14.23	X1.71	X2.43	X15.6	X127	X2.8	X3.06	X.28	X2.29	X5.64	X1.04	X3.92	X10
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<in
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	10
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	11
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	14
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	7
5	1	14.20	1.76	2.45	15.2	112	3.27	3.39	0.34	1.97	6.75	1.05	2.85	14
6	1	14.39	1.87	2.45	14.6	96	2.50	2.52	0.30	1.98	5.25	1.02	3.58	12

Os arquivos CSV podem ser lidos local ou remotamente usando a função read.table.

Salvando, apagando e carregando uma tabela

```r
save(wine, file="wine.RData", compress=TRUE)
```

```r
rm(wine)
```

```r
load("wine.RData")
head(wine, 3)
```

A data.frame: 3 × 14

	X1	X14.23	X1.71	X2.43	X15.6	X127	X2.8	X3.06	X.28	X2.29	X5.64	X1.04	X3.92	X10
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<ir
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	10
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	11
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	14

O R possui algumas funções para salvar e carregar variáveis como um arquivo binário.

Salvando em CSV

```r
write.table(wine, file="wine.csv", row.names=FALSE, quote=FALSE, sep=",")
```

```csv
X1,X14.23,X1.71,X2.43,X15.6,X127,X2.8,X3.06,X.28,X2.29,X5.64,X1.04,X3.92,X1065
1,13.2,1.78,2.14,11.2,100,2.65,2.76,0.26,1.28,4.38,1.05,3.4,1050
1,13.16,2.36,2.67,18.6,101,2.8,3.24,0.3,2.81,5.68,1.03,3.17,1185
1,14.37,1.95,2.5,16.8,113,3.85,3.49,0.24,2.18,7.8,0.86,3.45,1480
1,13.24,2.59,2.87,21,118,2.8,2.69,0.39,1.82,4.32,1.04,2.93,735
```

A função write.table pode salvar um arquivo CSV.

Filtrando tabela

```
i <- d$height > 1.7
i
```

TRUE · TRUE · FALSE · TRUE · TRUE · TRUE

```
d[i,]
```

A data.frame: 5 × 3

	weight	height	bmi
	<dbl>	<dbl>	<dbl>
1	60	1.75	19.59184
2	72	1.80	22.22222
4	90	1.90	24.93075
5	95	1.74	31.37799
6	72	1.91	19.73630

A filtragem de quadros de dados deve ser feita para o índice de linha do quadro de dados (antes da vírgula dentro dos colchetes).

Analisando desempenho em data.frame

```
rheight <- rnorm(100000, 1.8, sd=0.2)
rweight <- rnorm(100000, 72, sd=15)
```

```
start_time <- Sys.time()
hw <- data.frame(height=rheight, weight=rweight)
hw$bmi <- hw$weight/hw$height^2
end_time <- Sys.time()
end_time - start_time
```

```
Time difference of 0.002173424 secs
```

rnorm(n, m, s) gera n números aleatórios seguindo uma distribuição normal com média m e desvio padrão s
Sys.time() é usado para medir o tempo decorrido

Inclusão de atributo a partir de for loop

```
start_time <- Sys.time()
hw <- data.frame(height=rheight, weight=rweight)
for (i in 1:nrow(hw)) {
  hw$bmi[i] <- hw$weight[i]/hw$height[i]^2
}
end_time <- Sys.time()
end_time - start_time
```

```
Time difference of 3.977189 secs
```

A atribuição de valores para cada elemento na tabela é demorada.

Convertendo tabela para matriz, processando e voltando para tabela

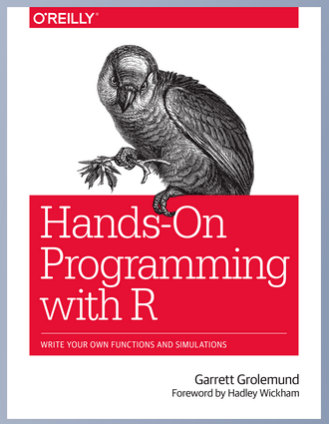
```
start_time <- Sys.time()
hw <- data.frame(height=rheight, weight=rweight)
hwm <- as.matrix(hw)
bmi <- 0
for (i in 1:nrow(hwm)) {
  bmi[i] <- hwm[i,1]/hwm[i,2]^2
}
hw$bmi <- bmi
end_time <- Sys.time()
end_time - start_time
```

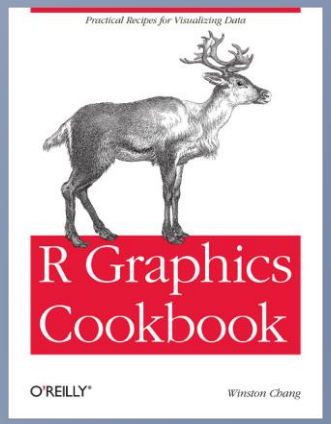
```
Time difference of 0.1461413 secs
```

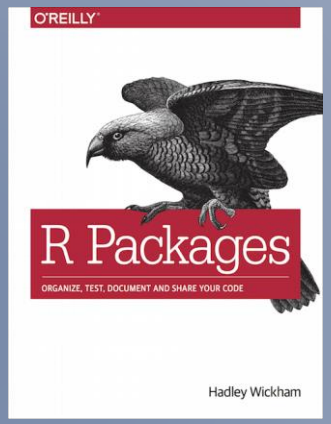
É melhor converter uma tabela em uma matriz e processá-lo como uma matriz.

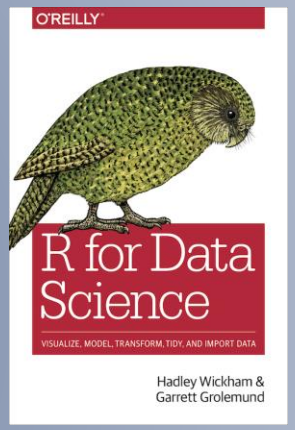
Referências

Código: https://github.com/eogasawara/R/blob/main/examples/10-Data-Frames.md







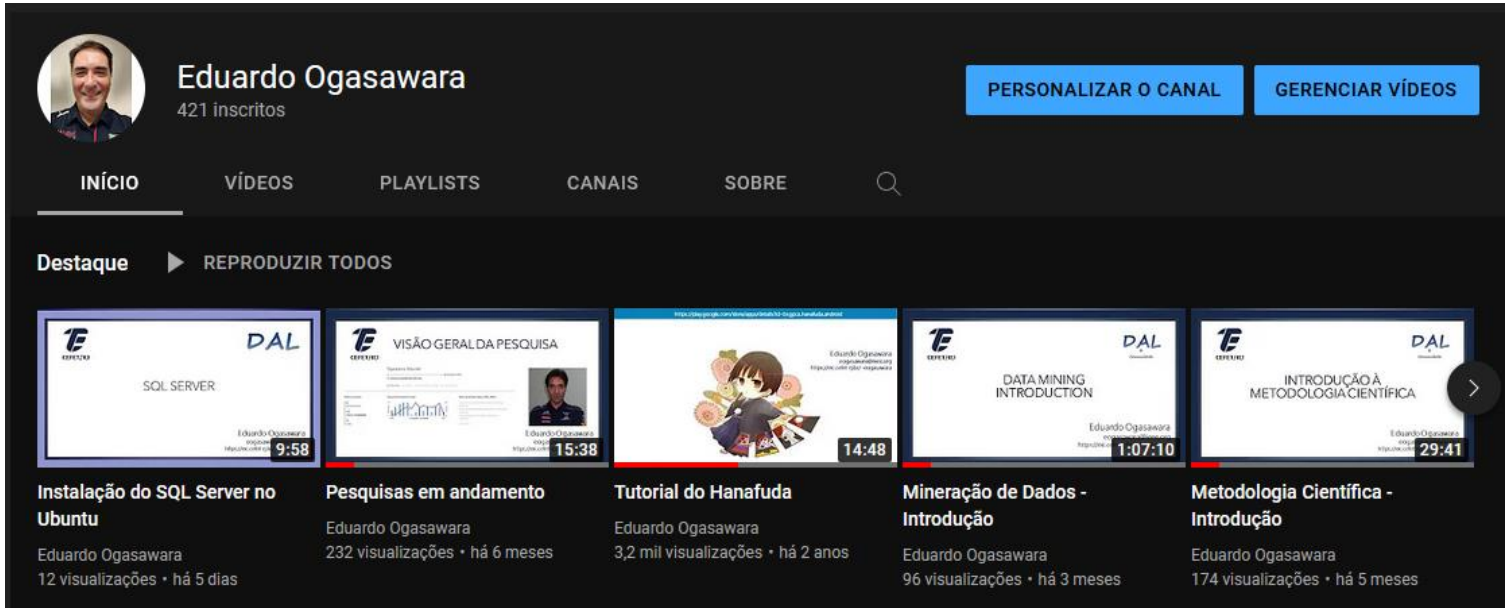


Hands-on Programming with R: https://rstudio-education.github.io/hopr/index.html
R Graphics Cookbook: https://r-graphics.org
R Packages: https://r-pkgs.org/index.html
R for Data Science: https://r4ds.had.co.nz

[1] G. Grolemund, 2014, Hands-On Programming with R: Write Your Own Functions and Simulations. O'Reilly Media, Inc.

Mantenha-se atualizado

Inscreva-se: https://eic.cefet-rj.br/~eogasawara/youtube



Slides and Jupyter notebooks: https://github.com/eogasawara/R